ISG-NIS-Helper — Installation

One zip, one folder, one command. No Python on the microscope PC, no network share, no separate build machine.

1. Install

1. Copy `ISG-NIS-Helper-<version>.zip` to the microscope PC — from a USB stick, the server, anywhere.
2. Unzip it into **`C:\ISG-NIS-Helper`**.
3. Double-click **`Run-PreScan.bat`**. It first asks what this instrument should be called in the logs (type a human name like `SCOPE-2 Ti2 Spinning Disk`, or press Enter to keep the computer name), then looks at the machine and writes `config.generated.ini`.
4. Read sections 4 and 5 of the report it prints (see below), then open `config.generated.ini` and check it.
5. Right-click **`install.bat`** → **Run as administrator**.
6. Open `users.csv` in that folder and add your trained users.

That is the whole procedure. Everything ISG-NIS-Helper needs lives in that one folder.

`install.bat` makes two changes to the machine: it registers the two scheduled tasks, and it repoints the existing NIS-Elements shortcuts (desktop, Start menu, taskbar, all user profiles) to open the sign-in window first. Each original shortcut is saved to `shortcuts.json` in the log store, and the uninstaller restores every one. Everything else is reversible by deleting a folder.

Where things live

`C:\ISG-NIS-Helper`	The program, `config.ini`, `users.csv`, the guides, the install logs
`C:\ProgramData\ISG-NIS-Helper`	Sessions, event logs, PIN overrides, failsafe state. Granted Modify to Users at install time

Nothing else, nowhere else. No share, no user profile, no registry keys.

`C:\ISG-NIS-Helper` is the recommended home: it is writable, so editing `config.ini` or `users.csv` never needs elevation. `C:\Program Files\ISG-NIS-Helper` also works and draws slightly less attention, but the cost is that **editing config or the user list there needs an elevated Notepad**, and a failed save is not self-explanatory. The rest of these guides assume `C:\ISG-NIS-Helper`.

Do not use a user profile such as `C:\Users\Nikon\AppData\Local`. It works on a machine with one shared login — which is the bad case, because the day somebody adds a second Windows account the agent silently fails to launch. And do not put a version number in the folder name: the scheduled tasks store the absolute path.

2. Reading the pre-scan

It reads the registry, file system and process list. **It changes nothing** — it writes only its three output files.

Output	What it is
`ISG-NIS-Helper-PreScan_<PC>_<date>.txt`	The readable report
`prescan.json`	The same, machine-readable
`config.generated.ini`	A config to review, then adopt

It also generates a random six-digit master PIN and prints it. **Write it down** — it is the manager's way in, and every pre-scan run makes a new one.

Two sections deserve your eyes.

Section 4, NIS-Elements. A full install puts a dozen executables in its folder — a settings utility, an AI trainer, a licence updater. Only the acquisition module matters; on a typical Advanced Research install that is `nis_ar.exe`

alone. The report shows its evidence: "currently running" and "shortcut" beat "found under Program Files", and it lists what it excluded from gating and why.

This matters more than it looks. A process name belonging to something permanently running makes ISG-NIS-Helper believe NIS is always open — sessions never end, every launch is flagged, and the watcher tries to close it on a loop. Preflight blocks the system services it knows about, but it cannot know every program on your machine.

Section 5, data roots. The scan counts `.nd2` files and proposes the folders holding most of them. If those cover under 80% it proposes whole drives instead — data filed as `<drive>:\<person>\<experiment>` has as many top folders as there are people. Drive roots like `D:\; E:\; F:\` are a perfectly normal answer. The system drive (`C:\`) is left out automatically — snapshotting it every session is slow and pulls in unrelated files.

Check what it proposes. If a data drive is missing, add it; if a proposed drive holds no acquisition data, drop it. (If anyone genuinely saves data onto `C:\`, add that specific folder by hand — the whole `C:\` drive is never proposed.)

Then open `config.generated.ini`. `computer_label` is the name you typed at the start of the scan; change it here if you want. A machine name like "DESKTOP-STQVARQ" tells a reader nothing, so give it something recognisable.

One rule: **comments must be on their own line.** A semicolon is the list separator inside `data_roots`.

When `install.bat` adopts the generated config it **preserves anything you already filled in by hand** that the generated file leaves blank, and tells you what it carried over.

3. The user list

`users.csv` in the ISG-NIS-Helper folder is the training gate. One list per machine, no network involved. The installer creates an empty one from the template if there isn't one; preflight fails a list with no active users, so an unfilled template cannot quietly reach an instrument.

See the Usage guide for the format and for adding people.

4. Verify at the instrument

Four checks, in order. The fourth matters most.

1. Click the NIS-Elements desktop icon → the sign-in window opens *first*. Sign in → NIS-Elements launches. Close NIS → a session file appears under `C:\ProgramData\ISG-NIS-Helper\logs`.
2. Click the icon and cancel the sign-in → NIS-Elements does not open.
3. Start NIS from its own executable, bypassing the icon, and do not sign in → the sign-in window comes to the front with a countdown and NIS closes after the grace period; `unauthorized_launch_closed` appears in that day's events.
4. **Run a long acquisition on a throwaway sample and leave it overnight.** In the morning the session is still open and the acquisition untouched.

Do not skip the fourth. It is the scenario where being wrong is most expensive.

One thing worth doing

Add `C:\ISG-NIS-Helper` to the Windows Defender exclusion list. PyInstaller output draws false positives, and a definition update quarantining an executable is the most likely way a Windows update stops ISG-NIS-Helper.

Clock synchronisation does not matter unless a machine is off by hours — both local time and UTC are recorded in every entry.

5. Removing it

Right-click **`uninstall.bat`** → **Run as administrator**. It restores every NIS-Elements shortcut it repointed, removes both scheduled tasks, stops both programs, clears `C:\ProgramData\ISG-NIS-Helper`, deletes the TEMP build workspace, and deletes the program folder itself.

It will not delete your session logs without asking, and it offers to copy them — along with `config.ini` and `users.csv` — to a dated folder on the Desktop first. Switches for when you already know: `-RemoveLogs`, `-KeepLogs`, `-KeepProgram`.

The program folder cannot delete itself while the script inside it is running, so that last step is handed to a detached shell that waits a few seconds after the window closes.

6. A stale copy is the easiest mistake here

`C:\ProgramData\ISG-NIS-Helper` is the **log store**. It is not a program folder. If a copy of the program ends up there — which is easy, because the installer's permission grant runs over that whole folder — you can find yourself running old code that looks like it works.

Three things now guard against it. `build.ps1` refuses outright to run from inside the log store. `install.bat` offers to delete stray program files it finds there. And both the build and the self-test print their version and their own path, so a stale run is obvious in the first three lines:

```
ISG-NIS-Helper self-test
  version : 3.0.4
  running : C:\ISG-NIS-Helper\selftest.py
```

If that path is not where you think you are, stop and look.

7. Starting over

```
schtasks /query /TN ISG-NIS-Helper-Agent      :: is it even installed?
```

Run `uninstall.bat` as administrator (section 5), then unzip the new version. The uninstaller handles the folders; you should not need to delete anything by hand.

Appendix: rebuilding after a code change

You only need this if you change the source. The zip already contains compiled executables that run on any Windows 10 or 11 machine without Python.

Build on any machine with Python — including a scope PC, which is what the build scripts assume. If the folder it runs from is not writable, it puts its workspace under TEMP and says so.

Python, if the machine has none: download the Windows 64-bit installer for a 3.11 or 3.12 release from python.org, and on the first screen **tick "Add python.exe to PATH"** before clicking anything. That one checkbox causes most of the confusing failures downstream.

Or with conda: `conda create -n isg-nis-helper python=3.12 -y`, then `conda activate isg-nis-helper`.

Then type `build.bat`. Not `python build.bat` — it is a batch file. If you use conda, type it in the activated Anaconda Prompt rather than double-clicking, because double-clicking opens a fresh shell with no environment and silently uses the wrong Python. `where python` should print a path inside `...\envs\isg-nis-helper`.

The build checks the environment first: it refuses the Microsoft Store placeholder, and it tests that tkinter actually works, because conda keeps Tcl/Tk somewhere PyInstaller has historically failed to find and the symptom is a clean build that dies at launch. If it fails, `conda install -n isg-nis-helper tk`.

It produces `ISG-NIS-Helper-<version>.zip` — the file you copy to every machine.

Build traps

Symptom	Fix
`python` opens the Microsoft Store	Settings → Apps → Advanced app settings → App execution aliases → turn **off** `python.exe` and `python3.exe`
"python is not recognized"	The PATH checkbox was missed; re-run the installer and choose Modify
pip fails with SSL errors	Ask IT for proxy settings. If pip is blocked, build elsewhere and copy the zip
PermissionError on a `.spec`	You are building in a non-writable folder; it redirects to TEMP, or build in `C:\ISG-NIS-Helper-build`

Appendix: config.ini

Key	Meaning
`[instrument] computer_label`	Human name of the scope
`[instrument] display_name`	What users see; never the project name
`[paths] nis_executable`	Full path to the acquisition program
`[paths] data_roots`	Semicolon-separated folders to snapshot
`[paths] users_file`	Blank = `users.csv` beside the program
`[paths] local_root`	Log store; leave alone
`[paths] network_root`	**Optional** folder to mirror finished logs into. Blank is a complete setup
`[processes] nis_process_names`	Acquisition modules only
`[behavior] launch_grace_seconds`	How long an unsigned launch may run
`[scan] exclude_dirs`	Skipped when snapshotting
`[access] master_pin`	Manager PIN for this instrument
`[failsafe] *`	See the Troubleshooting guide